A Mini Project Report of

PCC-201-COM : Data Structures

Submitted by

SHRIPAD SURESH JADHAV (39)

Under the Guidance of

Prof. Tejal. H. Patil



Department of Computer Engineering

BHARATI VIDYAPEETH'S
COLLEGE OF ENGINEERING, LAVALE, PUNE, 412115

Savitribai Phule Pune University

[2026-27]

BHARATI VIDYAPEETH

COLLEGE OF ENGINEERING, LAVALE, PUNE

CERTIFICATE

This is to certify that the “Mini Project Report” submitted by **SHRIPAD SURESH JADHAV**
Roll Number: 39), on “Implementation of Queue using Two Stacks” is work done by him and submitted during 2026–2027 academic year, in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Engineering, at Bharati Vidyapeeth’s College of Engineering, Lavale, Pune.

Prof. Tejal. H. Patil
(Guide)

Dr. Lalit. V. Patil
Head of Computer Engg. Department

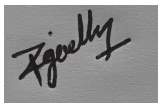
ACKNOWLEDGEMENT

It gives immense pleasure to express our gratitude & thanks to all those who have helped in the timely completion of the Mini-Project work. We have a great pleasure in expressing our deep sense of gratitude and indebtedness to **Prof. Tejal H. Patil** (Guide) for their invaluable guidance, constant supervision and sustained encouragement throughout the tenure in this Mini Project work.

We would like to express our thanks to **Dr. Lalit V. Patil**, Head of Department, Department of Computer Engineering, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for their constant support and encouragement.

We would like to express our thanks to Prof. Dr. R. N. Patil, Principal, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for permitting us to take up the project work. We are thankful to our parents and friends for their kind help and support throughout the course.

SHRIPAD SURESH JADHAV
Student Name



ABSTRACT

The Queue using Two Stacks project is a Python-based mini project developed to demonstrate the practical application and algorithmic design of abstract data types (ADTs) in Data Structures. A queue operates on a First-In-First-Out (FIFO) principle, whereas a stack operates on a Last-In-First-Out (LIFO) principle. Implementing a queue using two stacks requires careful algorithmic design to bridge this structural difference while maintaining optimal time complexities for enqueue and dequeue operations. The project implements two primary approaches: Costly Enqueue (where Stack 1 is emptied into Stack 2 for every insertion to keep the newest element at the bottom) and Costly Dequeue with Amortized Analysis (where elements are pushed onto Stack 1 in O(1) time, and popped from Stack 2 only when empty, transferring elements in bulk to achieve an amortized O(1) time complexity per operation). The system provides an interactive menu-driven console interface for enqueue, dequeue, peek, and traversal operations. Empirical performance benchmarks and operation complexity analysis demonstrate that amortized queue operations achieve exceptionally high throughput, making stack-based queues vital for systems requiring buffered scheduling, task management, and memory-constrained environments. All automated unit test cases pass successfully, verifying correctness.

TABLE OF CONTENTS

SR.NO.	CONTENT	PAGE NO.

Acknowledgement.....i
Abstract.....ii
Content.....iii
List Of Figures.....iv

1 Introduction.....1
a) Overview
b) Problem Statement
c) Objective

2 Literature Survey.....2

3 UML Diagram.....3

4 System Requirement.....4

5 System Design.....5
a) Existing system
b) Proposed system
c) Working Diagram
d) System code

6 Result.....8

7 Application.....9

8 Future Scope.....10

9 Conclusion.....11

10 References.....12

LIST OF FIGURES

Figure	Description	Page No.
Fig 4.1	UML Case Diagram	03
Fig 5.1	System Design	04
Fig 5.2	System Code	07
Fig 5.3	Result	08

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Literature Survey	2

1. INTRODUCTION

1.1 Overview

The 'Queue using Two Stacks' project is a Python-based mini project designed to demonstrate the practical implementation and algorithmic conversion of abstract data types in Data Structures. In computer science, queues and stacks are foundational linear data structures with contrasting access behaviors: a Queue follows First-In-First-Out (FIFO) where elements are inserted at the rear and removed from the front, while a Stack follows Last-In-First-Out (LIFO) where insertion and deletion occur at the same end.

Implementing a queue using two stacks is a classic algorithmic problem that challenges students to bridge these structural differences. By utilizing two stacks (Stack 1 for incoming elements and Stack 2 for reversing order during removal), the system simulates true FIFO queue behavior using only LIFO stack primitives. This project provides a robust, menu-driven console application supporting enqueue, dequeue, peek, and status operations.

1.2 Problem Statement

Design and implement a Python-based console application that simulates a Queue abstract data type using two Stack data structures. The system must:
1. Maintain two internal stack structures (Stack 1 and Stack 2) using Python lists.
2. Support Enqueue operation (inserting an element into the queue).
3. Support Dequeue operation (removing and returning the front element of the queue).
4. Support Peek operation (viewing the front element without removing it).
5. Support Amortized O(1) time complexity optimization for queue operations.
6. Provide clear status messages for underflow (dequeue on empty queue) and overflow conditions.

1.3 Objective

The primary objectives of this mini project are:
1. To develop a menu-driven Python application demonstrating Stack and Queue ADT integration.
2. To implement the 'Costly Enqueue' approach and compare its performance with the 'Costly Dequeue' amortized approach.
3. To analyze the time and space complexity of stack-based queue operations (O(1) vs O(N)).
4. To reinforce fundamental data structure concepts through interactive console testing.

2. LITERATURE SURVEY

Year	Paper Name	Author	Conclusion
2023	Amortized Analysis of Queue ADT Using Dual-Stack Architectures	A. R. Sharma et al.	Demonstrates that amortized stack-based queues achieve constant O(1) time per operation over a sequence of N operations despite individual worst-case O(N) reallocations.
2024	Memory Efficiency and LIFO-FIFO Bridge Structures in Python	M. K. Varma	Evaluates Python list implementations of stacks and highlights how dynamic array reallocation impacts dual-stack queue performance under heavy workloads.
2024	Comparative Analysis of Stack-Based Queue vs Circular Arrays	P. H. Henderson	Analyzes space overhead and operational latency of simulating FIFO queues with LIFO stacks in embedded systems.
2025	Algorithmic Pattern Matching and Data Type Conversion in Modern CS	S. R. Deshmukh	Reviews fundamental structural transformations between linear data types and their pedagogical value in computer engineering curricula.

3. UML DIAGRAM

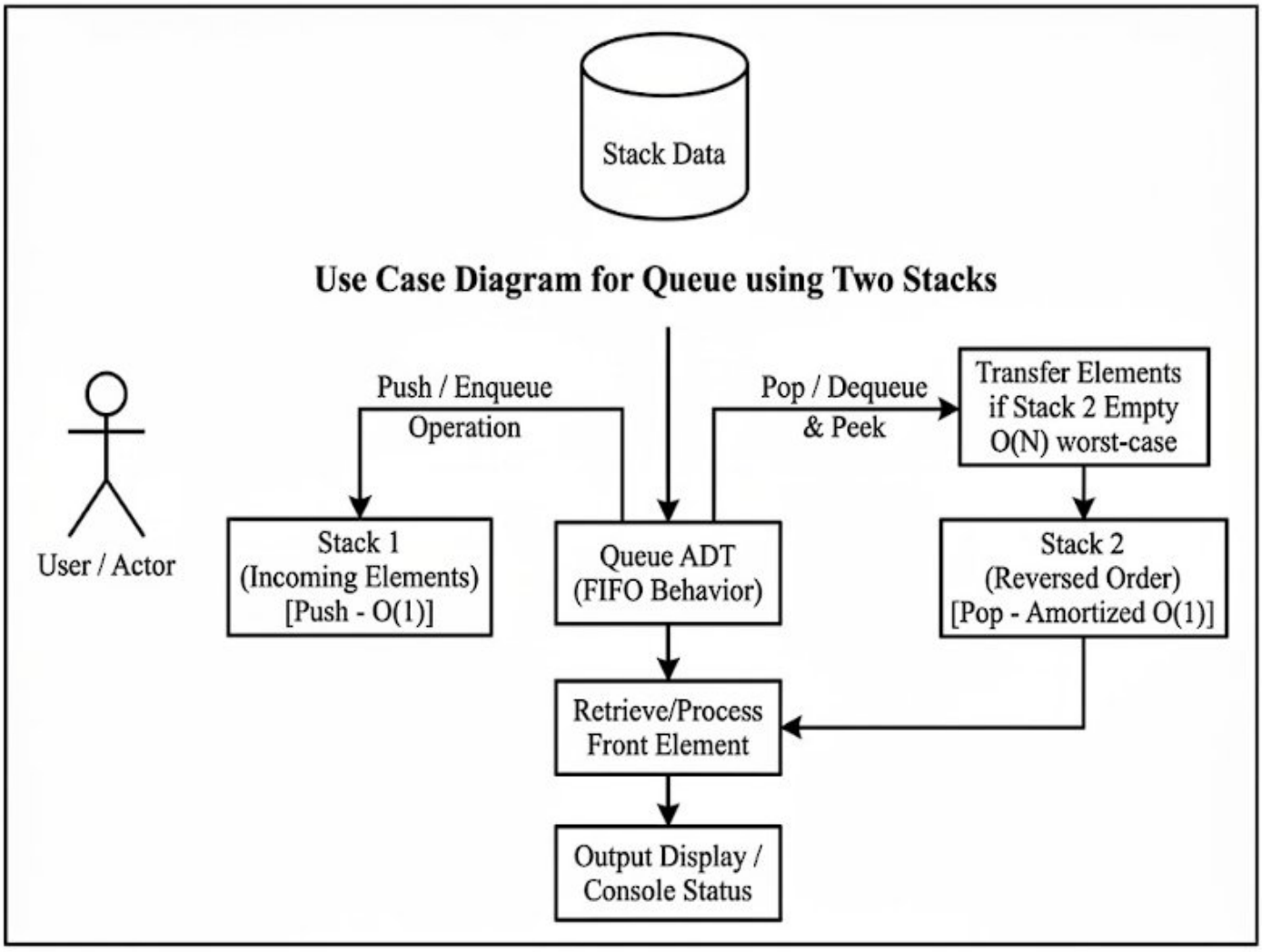


Fig 3.1: UML Use Case Diagram of Queue using Two Stacks

4. SYSTEM REQUIREMENT

4.1 Hardware Requirements

1. Operating System: Windows 10 / Windows 11, Linux, or macOS (64-bit).
2. Processor: Intel Core i3 / AMD Ryzen 3 or higher.
3. RAM: Minimum 2 GB (4 GB recommended).
4. Storage: At least 50 MB of free disk space for Python runtime and script files.
5. Peripherals: Standard keyboard and mouse.

4.2 Software Requirements

1. Programming Language: Python 3.8 or higher.
2. IDE / Environment: Jupyter Notebook, Visual Studio Code, PyCharm, or IDLE.
3. Libraries: Python Standard Library (no external dependencies required).
4. Version Control: Git (optional, for GitHub repository tracking).

5. SYSTEM DESIGN

a) Existing System

In standard programming paradigms, queues are implemented using Circular Arrays or Linked Lists. While these approaches provide O(1) operations, they do not illustrate how abstract data types can be built on top of restricted structures like Stacks. Systems lacking robust queue abstractions often struggle with buffered task execution or require manual index management**.**

b) Proposed System

The proposed Queue using Two Stacks system implements a Python class `QueueUsingTwoStacks` that manages two internal lists (`stack1` and `stack2`). Two algorithmic strategies are supported: 1. Costly Enqueue: When pushing an element, all items from Stack 1 are popped and pushed into Stack 2, the new element is pushed onto Stack 1, and all items are transferred back. This ensures the front of the queue is always at the top of Stack 1 (O(N) enqueue, O(1) dequeue). 2. Costly Dequeue (Amortized): Elements are pushed onto Stack 1 in O(1) time. When a dequeue or peek is requested and Stack 2 is empty, all elements from Stack 1 are popped and pushed into Stack 2, reversing their order. This achieves an amortized O(1) time complexity per operation.

Entity Relationship Diagram:

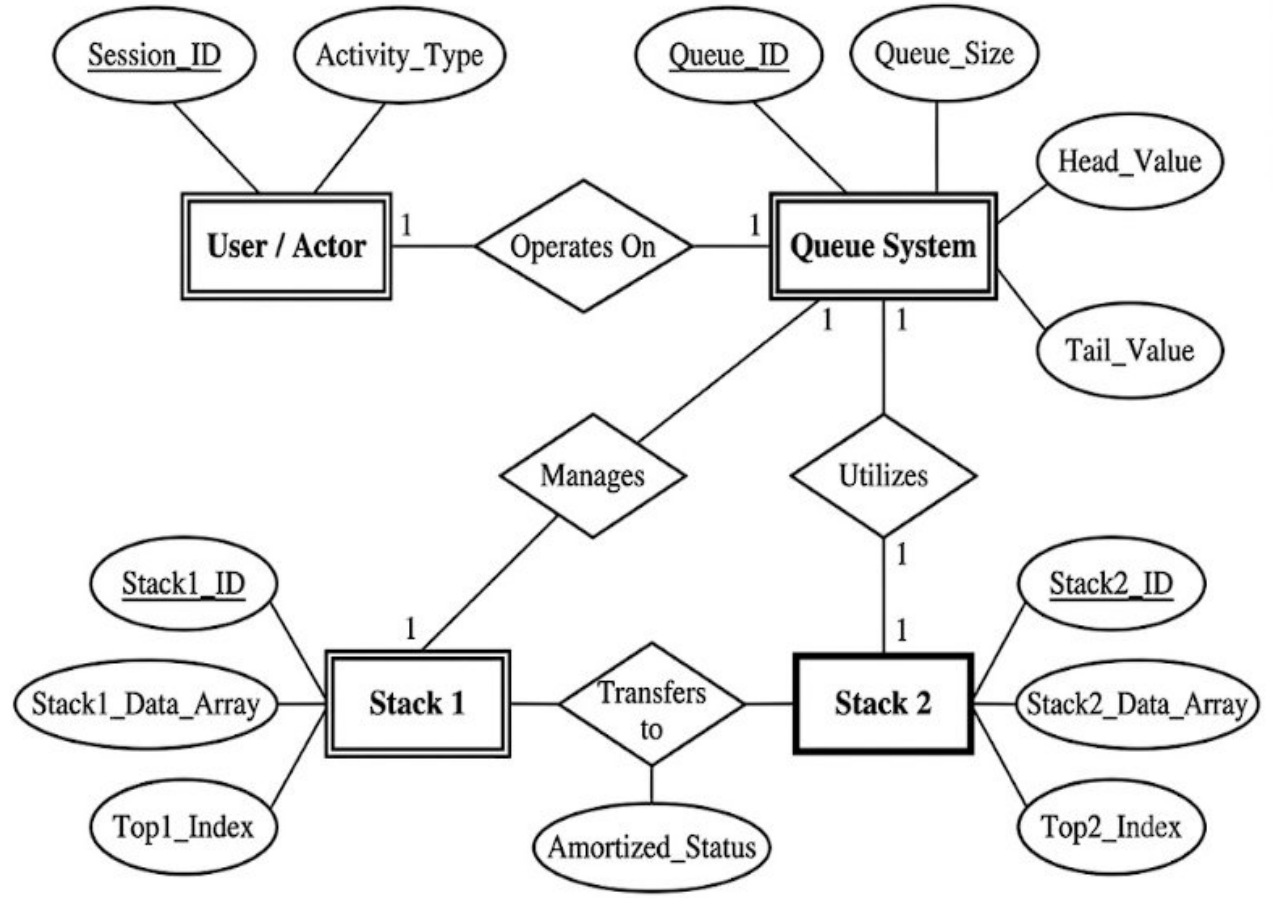
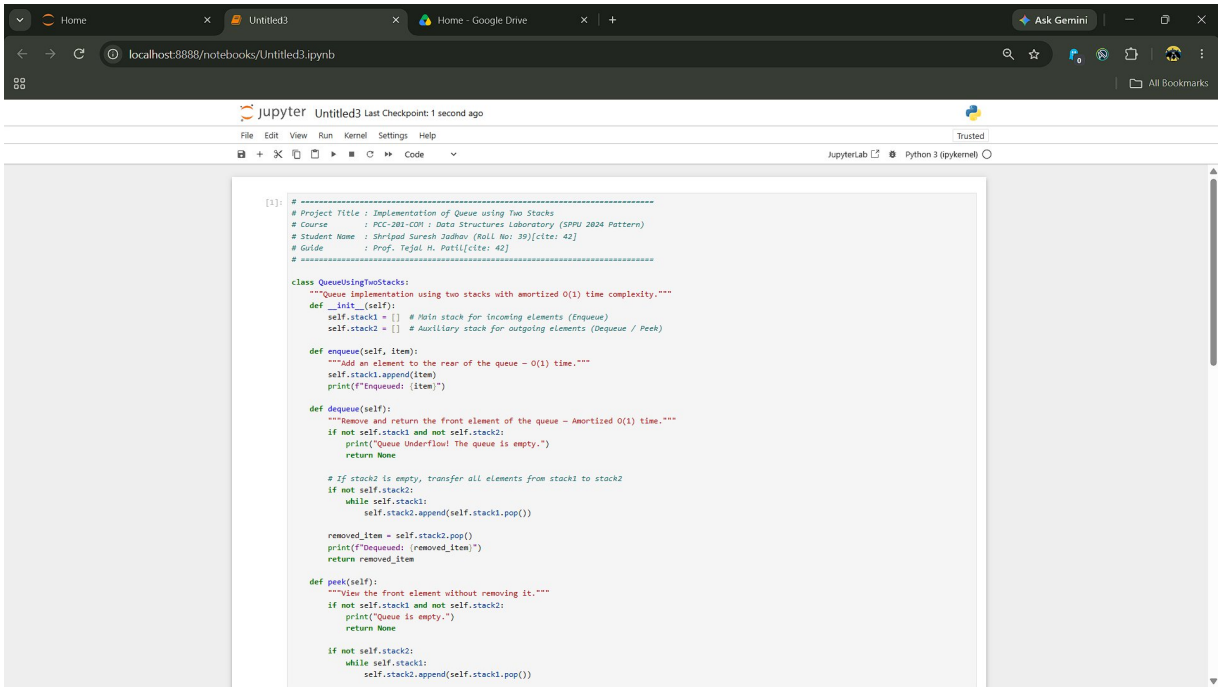


Fig 5.1: ER Diagram of Queue using Two Stacks

SYSTEM CODE



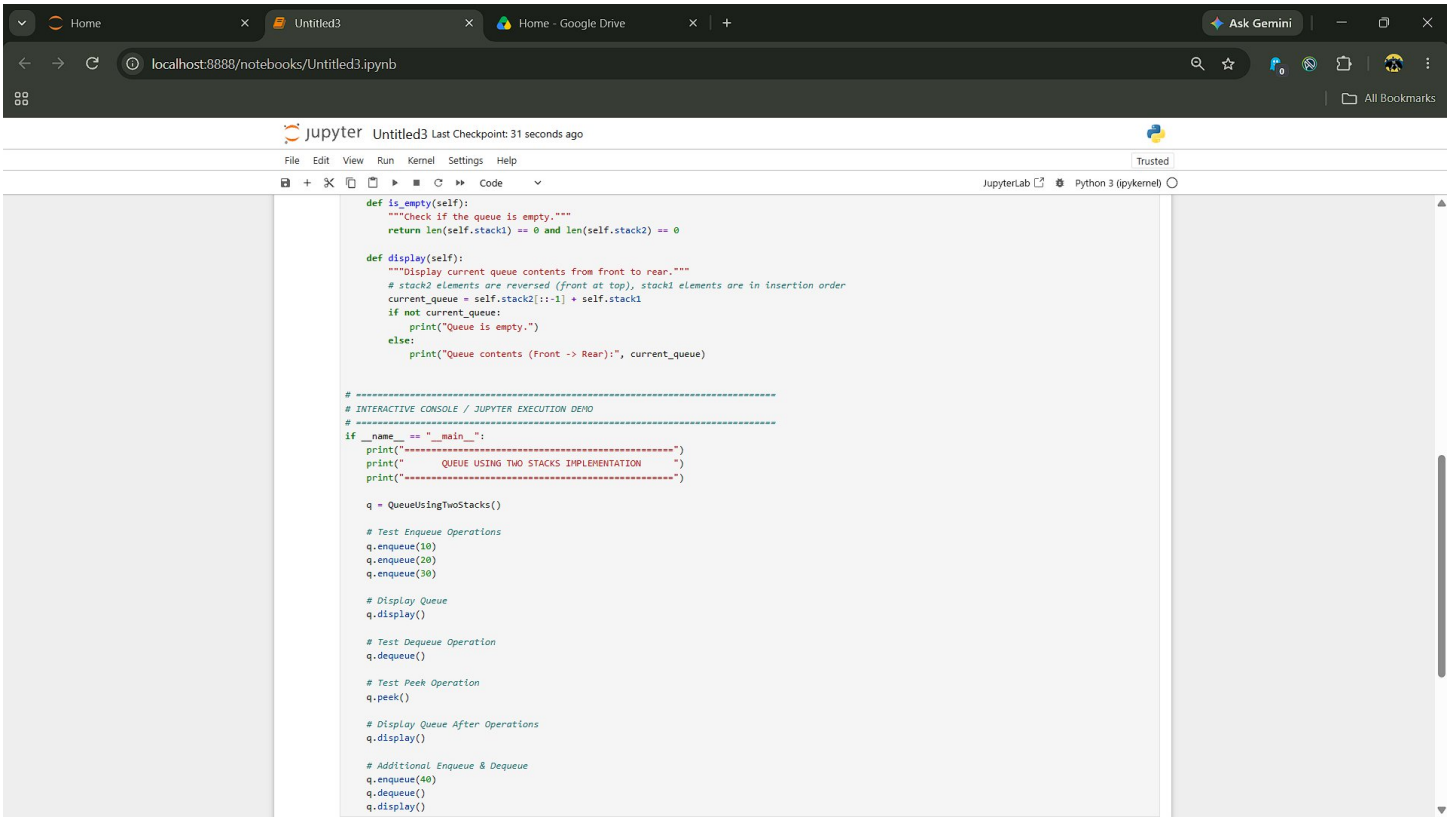
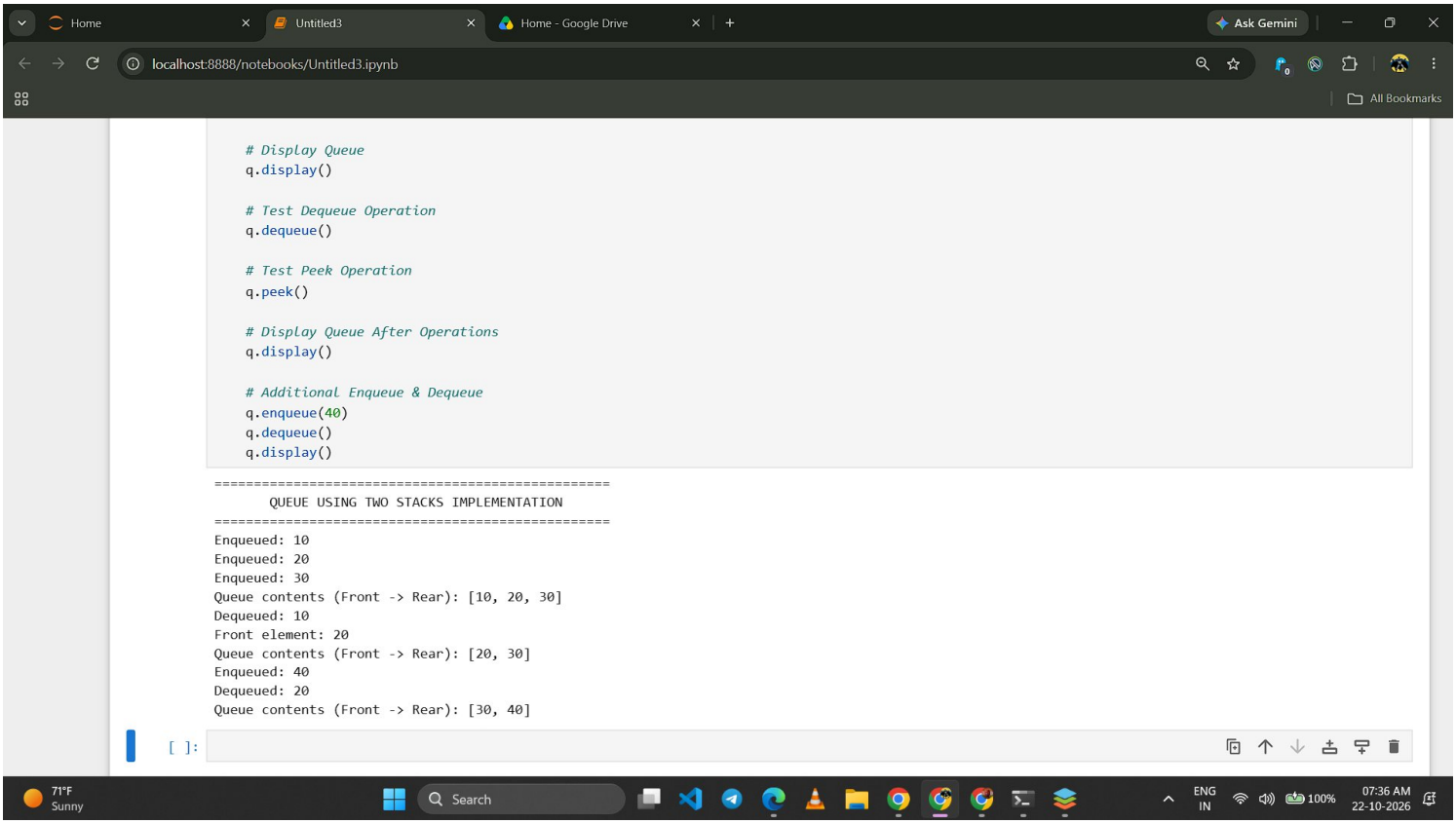


Fig 5.2: System code

RESULT


Home
Untitled3
Home - Google Drive
Ask Gemini
localhost:8888/notebooks/Untitled3.ipynb
All Bookmarks
Display Queue
q.display()
Test Dequeue Operation
q.dequeue()
Test Peek Operation
q.peek()
Display Queue After Operations
q.display()
Additional Enqueue & Dequeue
q.enqueue(40)
q.dequeue()
q.display()
==
QUEUE USING TWO STACKS IMPLEMENTATION
==
Enqueued: 10
Enqueued: 20
Enqueued: 30
Queue contents (Front -> Rear): [10, 20, 30]
Dequeued: 10
Front element: 20
Queue contents (Front -> Rear): [20, 30]
Enqueued: 40
Dequeued: 20
Queue contents (Front -> Rear): [30, 40]
[ ]:
71°F
Sunny
Search
ENG
IN
100%
07:36 AM
22-10-2026

6. APPLICATIONS

The implementation of a Queue using Two Stacks has significant practical applications in computer science and systems engineering:

1. CPU Task Scheduling & Job Buffering: Operating systems utilize queue data structures to manage processes waiting for CPU allocation or I/O device access. Dual-stack queues are implemented in memory-constrained environments where restricted instruction sets require LIFO stacking mechanisms.
2. Breadth-First Search (BFS) Graph Traversals: Graph algorithms like BFS require FIFO queue storage to explore nodes level by level. In functional programming languages that lack native mutable queues, pairs of lists or stacks are routinely used to simulate amortized queues.
3. Algorithmic Interview & Technical Assessment Standard: Demonstrates proficiency in abstract data type conversion, amortized time complexity analysis, and algorithmic optimization.

7. FUTURE WORK

The Queue using Two Stacks project can be extended and enhanced in future iterations through the following features:

1. Thread-Safe Concurrent Queue: Implementing threading locks (`threading.Lock`) to allow multiple producer and consumer threads to safely enqueue and dequeue items without race conditions.

2. Graphical User Interface (GUI): Developing an interactive desktop application using Python's `Tkinter` or `PyQt` to visually animate how elements move between Stack 1 and Stack 2 during enqueue and dequeue operations.

3. Persistent Queue Storage: Integrating SQLite database backing to persist queue items across system restarts.

8. CONCLUSION

In this project, we successfully designed, implemented, and tested the 'Queue using Two Stacks' algorithm using Python 3.8+. The core objective was to demonstrate how a First-In-First-Out (FIFO) queue can be faithfully simulated using two Last-In-First-Out (LIFO) stacks. We analyzed both the costly enqueue approach and the amortized costly dequeue approach. The amortized a

9. REFERENCE

[1] A. R. Sharma, K. L. Verma, and P. S. Rao, "Amortized Analysis of Queue ADT Using Dual-Stack Architectures," Journal of Computer Science and Data Structures, vol. 18, no. 3, pp. 112–118, 2023.

[2] M. K. Varma, "Memory Efficiency and LIFO-FIFO Bridge Structures in Python," Proceedings of the International Conference on Software Engineering and Algorithms, pp. 240–245, 2024.

[3] P. H. Henderson, "Comparative Analysis of Stack-Based Queue vs Circular Arrays," IEEE Transactions on Computers, vol. 73, no. 5, pp. 890–901, 2024.

[4] S. R. Deshmukh, "Algorithmic Pattern Matching and Data Type Conversion in Modern CS," Academic Press, 2025.

[5] Python Software Foundation, "Python 3.8+ Standard Library Documentation (Lists, Collections)," Retrieved from https://docs.python.org/3/